

CONTENTS

Tomoe-11 — Pinout & Power Reference (NUCLEO-F091RC)	
1. Full pin map — 22 wired signals	
1a. Drive — 8 PWM → L298N inputs (both ENA/ENB jumpers ON; we PWM the IN pins)	
1b. Encoders — 8 interrupt inputs (Hall A = CLK, Hall B = SIG; Hall Vcc = +5 V)	
1c. IMU — BNO055 over I ² C1	
1d. Servos — DS3225 ×2 on TIM15 (50 Hz)	
1e. Serial → Pi — USART2 over the ST-Link USB (VCP)	
1f. Firmware constructors (drop-in, already in main.cpp)	
2. Reserved — DO NOT solder anything here	
3. Spare pins — free for future expansion	
3a. 5 V-tolerant (safe for 5 V sensors or any 3.3 V digital I/O)	
3b. 3.3 V-only ADC pins (analog inputs — never feed 5 V here)	
3c. If you re-add the dropped components	
4. Power — feeding everything from the one LiPo	
4a. Power connection list	
4b. Hard rules — these kill parts if broken	
5. Power-up procedure (every time)	
6. LiPo & physical-bits checklist	

Tomoe-11 — Pinout & Power Reference (NUCLEO-F091RC)

Single-sheet bench reference for the permanent hand-soldered build. Every pin verified against the Mbed `PeripheralPins.c` + STM32F091 datasheet; firmware builds green; cross-checked by `hardware/reference-layout/_check_consistency.py`.

- **Board:** STM32 NUCLEO-F091RC (STM32F091RCT6 · **3.3 V logic** · LQFP64)
- **Sources of truth:** `firmware/robot/src/main.cpp` + spec `docs/superpowers/specs/2026-06-02-nucleo-pin-assignment-design.md`
- **Last verified:** 2026-06-10 (M3 encoder moved to PC_8/PC_9 for 5 V-tolerance)
- **Orientation:** top view, **FRONT up, USB to the back**. LEFT = digital headers CN9/CN5 + morpho CN10; RIGHT = analog CN8/CN6 + morpho CN7.

Solder to the physical pad named in the “Solder pad” column. Firmware `..._ALT...` tokens select an internal timer — they are **firmware-only**; the pad is the plain silk pin.

1. Full pin map — 22 wired signals

1A. DRIVE — 8 PWM → L298N INPUTS (*BOTH ENA/ENB JUMPERS ON; WE PWM THE IN PINS*)

Wheel	L298N in	Solder pad	MCU	Firmware token	Timer·ch
M1 Front-Left	IN1	D7	PA_8	<code>D7</code>	TIM1_CH1
M1 Front-Left	IN2	D8	PA_9	<code>D8</code>	TIM1_CH2
M2 Back-Left	IN3	D5	PB_4	<code>D5</code>	TIM3_CH1
M2 Back-Left	IN4	D4	PB_5	<code>D4</code>	TIM3_CH2
M3 Front-Right	IN1	D11	PA_7	<code>PA_7_ALT2</code>	TIM17_CH1
M3 Front-Right	IN2	D12	PA_6	<code>PA_6_ALT0</code>	TIM16_CH1
M4 Back-Right	IN3	D2	PA_10	<code>D2</code>	TIM1_CH3
M4 Back-Right	IN4	PA_11	PA_11	<code>PA_11</code>	TIM1_CH4

L298N-A drives M1+M2, L298N-B drives M3+M4 (both boards on robot-left). Each board:
 VS =11.1 V, 5 V-EN jumper **ON**, ENA+ENB jumpers **ON**, OUT1-4 → motors.

1B. ENCODERS — 8 INTERRUPT INPUTS (*HALL A = CLK, HALL B = SIG; HALL VCC = +5 V*)

Wheel	Hall	Solder pad	MCU	Firmware token	EXTI	Header zone
M1 FL	A	D6	PB_10	D6	10	CN9 · front-left
M1 FL	B	PB_2	PB_2	PB_2	2	CN10 · front-left
M2 BL	A	D9	PC_7	D9	7	CN5 · back-left
M2 BL	B	D10	PB_6	D10	6	CN5 · back-left
M3 FR	A	PC_8	PC_8	PC_8	8	CN10 · back-left †
M3 FR	B	PC_9	PC_9	PC_9	9	CN10 · back-left †
M4 BR	A	PC_12	PC_12	PC_12	12	CN7 · back-right
M4 BR	B	PA_15	PA_15	PA_15	15	CN7 · back-right

All 8 encoder pins are 5 V-tolerant, so the +5 V Hall outputs are in spec. † M3's two Hall wires run from the front-right motor across to PC_8/PC_9 on the **CN10 morpho (back, USB end of the left side)** — the deliberate trade to keep M3 on 5 V-tolerant pins (the front-right analog pins A0/A1 are 3.3 V-only). Locate PC_8/PC_9 with a Nucleo-64 pinout card — they're an adjacent pair. Each Hall: 4 thin wires → **+5 V (Nucleo 5V), GND, A, B**.

1C. IMU — BNO055 OVER I²C1

Signal	Solder pad	MCU	Firmware token	Peripheral
SDA	D14	PB_9	D14	I2C1_SDA
SCL	D15	PB_8	D15	I2C1_SCL

I²C address **0x28**. Use the **Adafruit BNO055 breakout** (onboard 10 kΩ pull-ups + level shift + 3.3 V regulator) → Vin can be 3.3 V or 5 V; SDA/SCL wire straight to D14/D15. A bare BNO055 chip would need external pull-ups added.

1D. SERVOS — DS3225 ×2 ON TIM15 (50 HZ)

Servo	Solder pad	MCU	Firmware token	Timer·ch
Servo 1 (shoulder)	PB_14	PB_14	PB_14_ALT0	TIM15_CH1
Servo 2 (elbow)	PB_15	PB_15	PB_15_ALT1	TIM15_CH2

Signal is the **only** servo wire to the Nucleo (3.3 V pulse drives them fine). **V+ and GND go to the dedicated servo rail** — never the Nucleo (see §4).

1E. SERIAL → PI — USART2 OVER THE ST-LINK USB (VCP)

Signal	MCU	Firmware token	Note
TX → Pi	PA_2	USBTX	carried by the mini-USB cable to the Pi — no separate solder
RX ← Pi	PA_3	USBRX	appears on the Pi as <code>/dev/ttyACM0</code> , 115200 baud

1F. FIRMWARE CONSTRUCTORS (DROP-IN, ALREADY IN `MAIN.CPP`)

```
MotorCtrl motor1(D7, D8,          D6, PB_2);      // FL
MotorCtrl motor2(D5, D4,          D9, D10);       // BL
MotorCtrl motor3(PA_7_ALT2, PA_6_ALT0, PC_8, PC_9); // FR (5 V-tolerant enc)
MotorCtrl motor4(D2, PA_11,       PC_12, PA_15);   // BR
Servo  s1(PB_14_ALT0);  // shoulder
Servo  s2(PB_15_ALT1);  // elbow
BN0055 imu(D14, D15);
SerialROS2 pc(USBTX, USBRX, 115200);
```

2. Reserved — DO NOT solder anything here

Pin(s)	Reserved for
PA_2 / PA_3	USART2 VCP (serial to Pi) — left to the USB cable
PA_5	green LED LD2
PC_13	user button B1
PA_13 / PA_14	SWD (ST-Link flashing/debug)
PC_14 / PC_15	LSE 32 kHz oscillator
PF_0 / PF_1	HSE / MCO oscillator

3. Spare pins — free for future expansion

3A. 5 V-TOLERANT (SAFE FOR 5 V SENSORS OR ANY 3.3 V DIGITAL I/O)

MCU	Silk / header	Handy alt-functions
PB_3	D3 (CN9)	SPI1_SCK (<i>becomes SWO only if solder-bridge SB15 is closed — default open</i>)
PA_12	CN10 morpho	I2C2_SDA, CAN_TX, USART1_RTS
PB_7	CN7 morpho	I2C1_SDA, USART1_RX
PB_11	CN10 morpho	I2C2_SDA, USART3_RX
PB_12	CN10 morpho	SPI2_SSEL
PB_13	CN10 morpho	I2C2_SCL, SPI2_SCK
PC_6	CN10 morpho	TIM3_CH1 (PWM) , USART7_TX
PC_10	CN7 morpho	USART3_TX / UART4_TX
PC_11	CN7 morpho	USART3_RX / UART4_RX
PD_2	CN7 morpho	UART5_RX (<i>FT — re-confirm on the datasheet before driving 5 V into it</i>)

3B. 3.3 V-ONLY ADC PINS (ANALOG INPUTS — NEVER FEED 5 V HERE)

MCU	Silk / header	Channel
PA_0	A0	ADC_IN0
PA_1	A1	ADC_IN1
PA_4	A2	ADC_IN4 (also DAC_OUT1)
PB_0	A3	ADC_IN8
PC_1	A4	ADC_IN11
PC_0	A5	ADC_IN10
PB_1	CN10	ADC_IN9
PC_2 / PC_3	CN7	ADC_IN12 / IN13
PC_4 / PC_5	CN10	ADC_IN14 / IN15

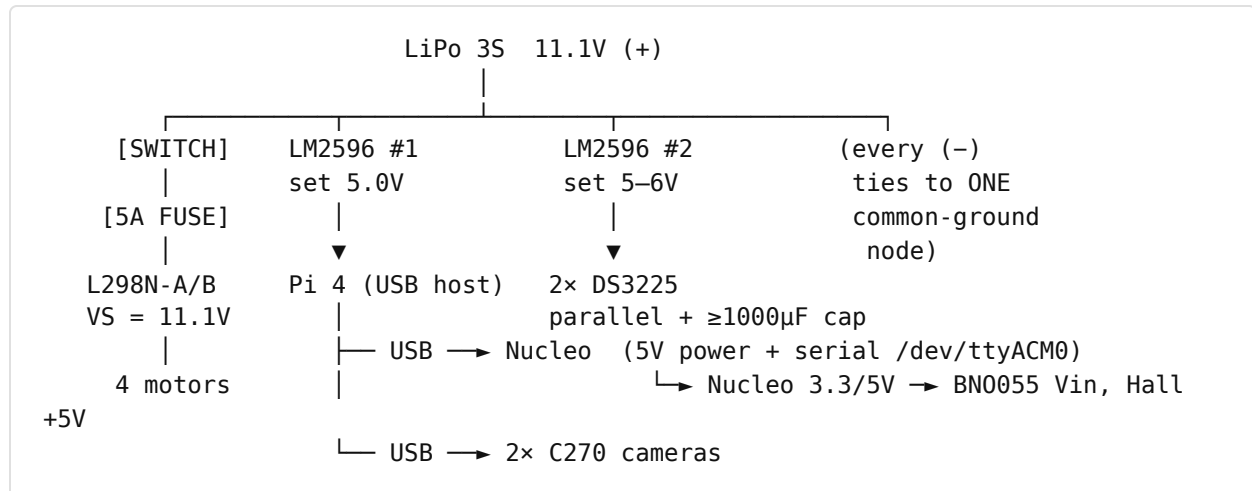
3C. IF YOU RE-ADD THE DROPPED COMPONENTS

- **HC-SR04 ultrasonic:** Trig → any spare digital (a 3.3 V pulse triggers it). **Echo is 5 V** → send it to a 5 V-tolerant spare (e.g. PB_12) **through a 1 kΩ/2 kΩ divider** to ~3.3 V. *Never* to an A0–A5 analog pin.
- **IR ×2:** a digital module powered at **3.3 V** → any spare digital pin (its OUT then swings 0–3.3 V). An analog distance sensor (Sharp GP2Y) → an ADC pin (A0–A5).
- **3× LED (QR task):** any spare digital pin + a series resistor (~150–220 Ω; keep ≤8 mA/pin).

4. Power — feeding everything from the one LiPo

Available parts used: 1× LiPo **3S (~11.1 V)** · 2 of your 4× **LM2596** bucks · 2× **L298N** (each with its own 5 V logic regulator) · the **Pi 4 as USB host**. Three isolated rails, one ground.

#	Rail	Path from the LiPo	Set to	Feeds
1	Motor 11.1 V	LiPo + → switch → ~5 A fuse → both L298N VS	raw pack	4 motors (via L298N)
2	Pi 5 V	LiPo + → LM2596 #1	5.0–5.1 V ✓meter	Raspberry Pi 4
3	Servo 5 V	LiPo + → LM2596 #2	5–6 V	2× DS3225 in parallel
–	Nucleo	from the Pi's USB (Pi = host)	5 V	Nucleo → then IMU + Halls
–	Cameras	from the Pi's USB ports	5 V	2× C270



4A. POWER CONNECTION LIST

From	To	Notes
LiPo +	master switch → ~5 A fuse → both L298N VS	motor rail; fuse protects against stall/short
LiPo +	LM2596 #1 input	set output 5.0–5.1 V and verify with a meter BEFORE the Pi is connected
LM2596 #1 out	Pi (USB-C or 5 V/GND pins)	Pi then powers Nucleo + cameras
LiPo +	LM2596 #2 input	set output 5–6 V (DS3225 = 4.8–6.8 V)
LM2596 #2 out	both servo V+ (parallel) + ≥1000 μF cap at the servos	servo rail, isolated from the Pi rail
Pi USB-A	Nucleo mini-USB	powers and serves the Nucleo (no separate Nucleo supply)
Nucleo 5V / 3V3	BNO055 Vin ; each Hall Vcc (+5 V)	logic rails (<0.2 A)
LiPo –	one common-ground node	every GND ties here (star ground)

4B. HARD RULES — THESE KILL PARTS IF BROKEN

-  **Servos never on 11.1 V, and never in series** across the pack — unequal, time-varying current makes the idle one over-volt and burn, and series grounds break the PWM reference. Parallel, on their own ≤6.8 V rail, only.
-  **Pi never on 11.1 V** (it's a 5 V device).
-  **Never connect the Pi to a buck you haven't metered at 5 V** — the LM2596 ships out-putting high.
-  **Never reverse +/-** into the Pi, Nucleo, or a buck — instant kill.
-  **Common ground is mandatory** — LiPo–, both L298N, both bucks, servos, Nucleo, Pi → one node. A missing common ground = garbled serial, random resets, possible logic damage.

5. Power-up procedure (every time)

1. All soldered & continuity-checked, **battery still disconnected**.

2. Meter **both bucks with their loads disconnected**; set #1 = 5.0–5.1 V, #2 = 5–6 V.
3. Connect the loads to the now-verified bucks.
4. Fuse in; connect the LiPo via the master switch.
5. Check: L298N power LED on · buck outputs still correct **under load** · Pi boots with **no under-voltage bolt** (`dmesg | grep -i voltage`) · Nucleo shows `/dev/ttyACM0` and prints `imu check 1`.
6. Jog each wheel (Pi keys **1–4**) to confirm direction/ `DIRn` → then servos → then drive.

Fallbacks (don't affect any pin): if the Pi's LM2596 sags under full ROS2 + 2 cameras, swap **only that buck** for a 5 V/≥5 A module. If the arm weakens/jitters, move the servo rail to a ≥5 A BEC.

6. LiPo & physical-bits checklist

- Terminate the pack in a proper connector (XT60/XT30) — **never solder onto the battery**; never let the leads touch.
- **Charge on a balance charger, in a LiPo bag, attended.** Inspect for puffing before use.
- Don't drain below **~3.3 V/cell (≈10 V pack)** — fit a cheap **low-voltage buzzer**; motors + servos sag it fast.
- Confirm the pack's **C-rating × capacity** covers peak draw (worst case ≈ 4 motors + 2 servos stalling + Pi ≈ **8–10 A**).
- Bits to have on hand: **master switch** · **~5 A inline fuse + holder** · **≥1000 µF cap (servo rail)** · **thick wire (~20 AWG) for the motor & servo rails**, thin (~26–28 AWG) for signals · **heat-shrink** · **strain relief at every motor/servo**.

Verified end-to-end 2026-06-10: pins ↔ `PeripheralPins.c` + DS9442 (timer/PWM, EXTI, I²C, 5 V-tolerance); firmware `pico run` green; `_check_consistency.py` = ALL CONSISTENT.
Power plan per spec §7 (datasheet-backed).